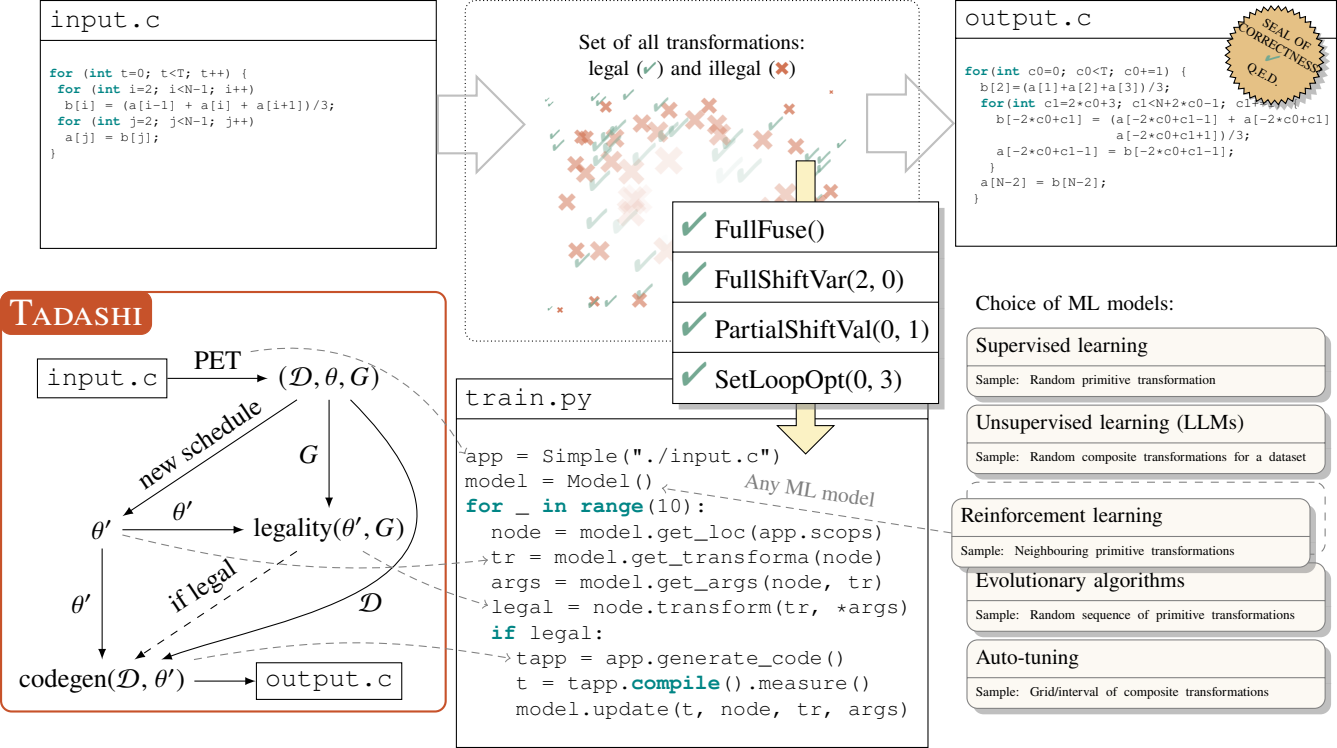
input.c
for (int t=0; t<T; t++) {
for (int i=2; i<N-1; i++)
b[i] = (a[i-1] + a[i] + a[i+1])/3;
for (int j=2; j<N-1; j++)
a[j] = b[j];
}
Set of all transformations:
legal (✔) and illegal (✖)
output.c
SEAL OF CORRECTNESS
Q.E.D.
for(int c0=0; c0<T; c0+=1) {
b[2]=(a[1]+a[2]+a[3])/3;
for(int c1=2*c0+3; c1<N+2*c0-1; c1+=1)
b[-2*c0+c1] = (a[-2*c0+c1-1] + a[-2*c0+c1]
a[-2*c0+c1+1])/3;
a[-2*c0+c1-1] = b[-2*c0+c1-1];
}
a[N-2] = b[N-2];
}
✔ FullFuse()
✔ FullShiftVar(2, 0)
✔ PartialShiftVal(0, 1)
✔ SetLoopOpt(0, 3)
TADASHI
input.c
PET
$(\mathcal{D}, \theta, G)$
new schedule
G
θ'
θ'
legality(θ', G)
θ'
if legal
$\mathcal{D}$
codegen$(\mathcal{D}, \theta')$
output.c
train.py
app = Simple("./input.c")
model = Model()
for _ in range(10):
node = model.get_loc(app.scops)
tr = model.get_transforma(node)
args = model.get_args(node, tr)
legal = node.transform(tr, *args)
if legal:
tapp = app.generate_code()
t = tapp.compile().measure()
model.update(t, node, tr, args)
Any ML model
Choice of ML models:
Supervised learning
Sample: Random primitive transformation
Unsupervised learning (LLMs)
Sample: Random composite transformations for a dataset
Reinforcement learning
Sample: Neighbouring primitive transformations
Evolutionary algorithms
Sample: Random sequence of primitive transformations
Auto-tuning
Sample: Grid/interval of composite transformations